

IMPORTANT LAB TASK FROM LAB 01 TO 06

LAB 01:

Lab 1

- **Directory Structure:** Create folders a, b, and c in a parent-child relationship (e.g., a/b/c).
- **File Management:** * In folder c, create 3 text files.
 - Write content into the first file.
 - Merge the contents of File 1 and File 2.
 - Edit File 2.

```

Apr 3 07:25
ubuntu@ubuntu: ~/a/b/c

ubuntu@ubuntu:~/Desktop$ ls
firstfolder  secondfolder
folder3      ubuntu-desktop-bootstrap_ubuntu-desktop-bootstrap.desktop
ubuntu@ubuntu:~/Desktop$ d
d: command not found
ubuntu@ubuntu:~/Desktop$ cd
ubuntu@ubuntu:~$ ls
Desktop  Documents  Downloads  Music  Pictures  Public  Templates  Videos  snap
ubuntu@ubuntu:~$ mkdir -p a/b/c
ubuntu@ubuntu:~$ ls
Desktop  Downloads  Pictures  Templates  a
Documents  Music      Public    Videos    snap
ubuntu@ubuntu:~$ cd a
ubuntu@ubuntu:~/a$ ls
b
ubuntu@ubuntu:~/a$ cd b
ubuntu@ubuntu:~/a/b$ ls
c
ubuntu@ubuntu:~/a/b$ cd c
ubuntu@ubuntu:~/a/b/c$ cat > file1.txt
this is file 1
ubuntu@ubuntu:~/a/b/c$ cat > file2.txt
This is file 2
ubuntu@ubuntu:~/a/b/c$ cat > file3.txt
this is file 3
ubuntu@ubuntu:~/a/b/c$ ls
file1.txt  file2.txt  file3.txt
ubuntu@ubuntu:~/a/b/c$ cat file1.txt file2.txt > merge.txt
ubuntu@ubuntu:~/a/b/c$ ls
file1.txt  file2.txt  file3.txt  merge.txt
ubuntu@ubuntu:~/a/b/c$ cat merge.txt
this is file 1
This is file 2
  
```

```

ubuntu@ubuntu:~/a/b/c$ cat merge.txt
this is file 1
This is file 2
ubuntu@ubuntu:~/a/b/c$ cat >> merge.txt
This is edit file
ubuntu@ubuntu:~/a/b/c$ cat merge.txt
this is file 1
This is file 2
This is edit file

```

LAB 02:

Lab 2

- **Basic Linux Commands:** * Create 3 files using touch: CSE, SWE, and CIS.
 - Write text into each file.
 - Copy CSE to SWE.
 - Move SWE to CIS.
- **Search and Compression:**
 - Search for a specific word inside the CIS file.
 - Use tar to archive CIS, then extract the .tar file.
 - Use gzip to compress CIS, then gunzip to decompress it.

```

Apr 3 07:35
ubuntu@ubuntu: ~/l2

ubuntu@ubuntu:~$ ls
Desktop  Documents  Downloads  Music  Pictures  Public  Templates  Videos  a  snap
ubuntu@ubuntu:~$ mkdir l2
ubuntu@ubuntu:~$ ls
Desktop  Documents  Downloads  Music  Pictures  Public  Templates  Videos  a  l2  snap
ubuntu@ubuntu:~$ cd l2
ubuntu@ubuntu:~/l2$ ls
ubuntu@ubuntu:~/l2$ touch cse swe cis
ubuntu@ubuntu:~/l2$ ls
cis  cse  swe
ubuntu@ubuntu:~/l2$ nano cse
ubuntu@ubuntu:~/l2$ nano swe
ubuntu@ubuntu:~/l2$ nano cis
ubuntu@ubuntu:~/l2$ cp cse swe
ubuntu@ubuntu:~/l2$ nano swe
ubuntu@ubuntu:~/l2$ mv swe cis
ubuntu@ubuntu:~/l2$ ls
cis  cse
ubuntu@ubuntu:~/l2$ nano cis
ubuntu@ubuntu:~/l2$

```

```

ubuntu@ubuntu: ~/l2
ubuntu@ubuntu:~/l2$ ls
cis  cse
ubuntu@ubuntu:~/l2$ tar -cvf backup cis cse
cis
cse
ubuntu@ubuntu:~/l2$ ls
backup  cis  cse
ubuntu@ubuntu:~/l2$ tar -cvf backup.tar cis cse
cis
cse
ubuntu@ubuntu:~/l2$ ls
backup  backup.tar  cis  cse
ubuntu@ubuntu:~/l2$ rm -rf cis cse
ubuntu@ubuntu:~/l2$ ls
backup  backup.tar
ubuntu@ubuntu:~/l2$ rm -rf backup
ubuntu@ubuntu:~/l2$ ls
backup.tar
ubuntu@ubuntu:~/l2$ tar -xvf backup.tar
cis
cse
ubuntu@ubuntu:~/l2$ ls
backup.tar  cis  cse
ubuntu@ubuntu:~/l2$ gzip cse
ubuntu@ubuntu:~/l2$ ls
backup.tar  cis  cse.gz
ubuntu@ubuntu:~/l2$ gzip -d cse.gz
ubuntu@ubuntu:~/l2$ ls
backup.tar  cis  cse
ubuntu@ubuntu:~/l2$

```

LAB 03:

Lab 3

- **Password Masking:** Hide a password based on the length of the input.
- **Arrays:** Using EOF for input array then print all the array element.

```

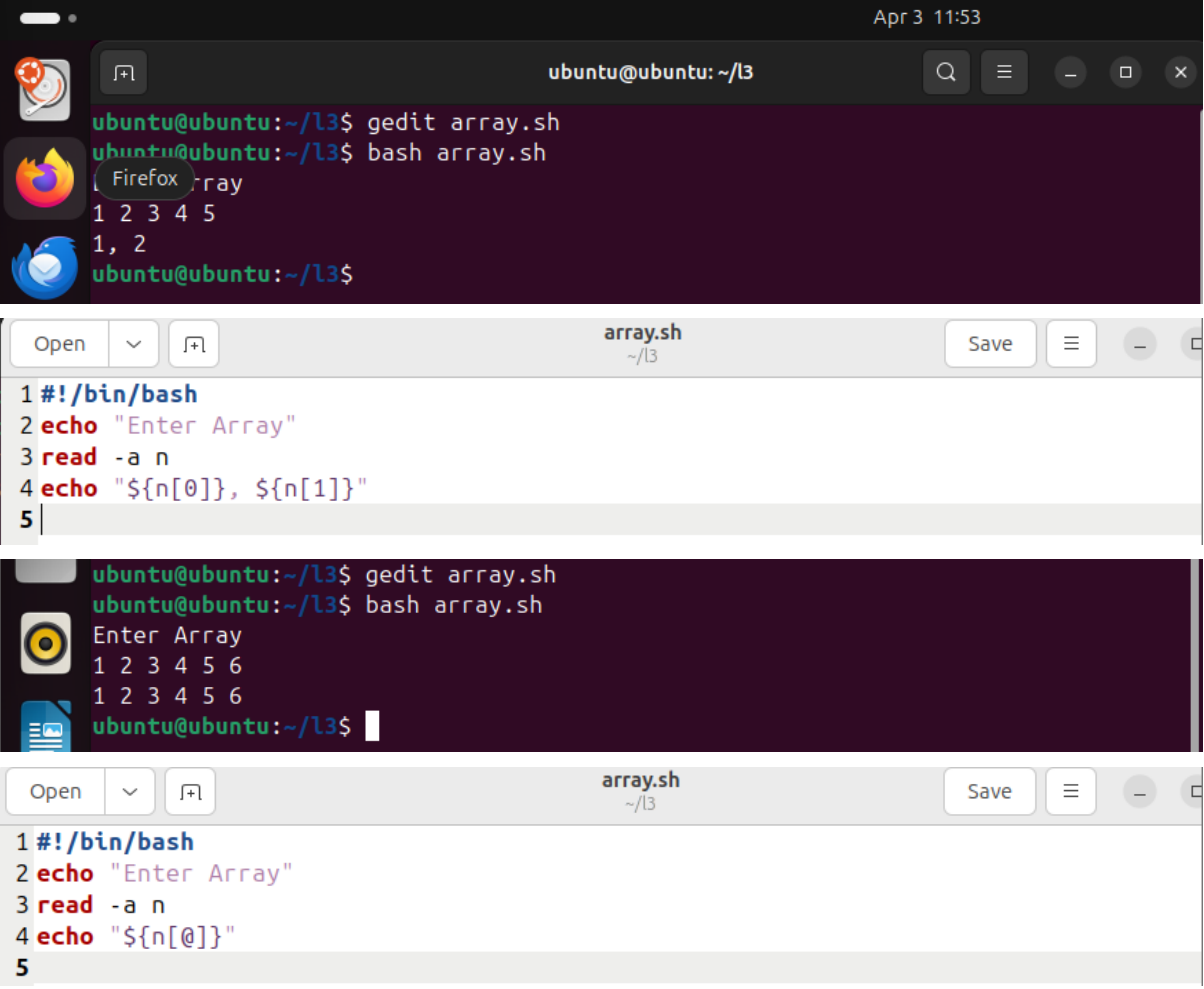
ubuntu@ubuntu: ~/l3
ubuntu@ubuntu:~/l3$ gedit prothom.sh
ubuntu@ubuntu:~/l3$ bash prothom.sh
Enter Password:1234512
ubuntu@ubuntu:~/l3$

```

```

prothom.sh
~/l3
1 #!/bin/bash
2 read -sp "Enter Password:" pass
3 echo "$pass"

```



The image shows two screenshots of a Linux environment. The top screenshot shows a terminal window where a user creates a file `array.sh` using `gedit` and runs it with `bash array.sh`. The script prompts for an array, and the user enters `1 2 3 4 5`. The script outputs `1, 2`. The bottom screenshot shows the same terminal window where the user runs `array.sh` again, enters `1 2 3 4 5 6`, and the script outputs `12 3 4 5 6`. The code editor shows the script content: `#!/bin/bash`, `echo "Enter Array"`, `read -a n`, and `echo "${n[@]}"`.

```

ubuntu@ubuntu: ~/l3
ubuntu@ubuntu:~/l3$ gedit array.sh
ubuntu@ubuntu:~/l3$ bash array.sh
Enter Array
1 2 3 4 5
1, 2
ubuntu@ubuntu:~/l3$

array.sh
~/l3
1 #!/bin/bash
2 echo "Enter Array"
3 read -a n
4 echo "${n[@]}"
5

ubuntu@ubuntu:~/l3$ gedit array.sh
ubuntu@ubuntu:~/l3$ bash array.sh
Enter Array
1 2 3 4 5 6
12 3 4 5 6
ubuntu@ubuntu:~/l3$

array.sh
~/l3
1 #!/bin/bash
2 echo "Enter Array"
3 read -a n
4 echo "${n[@]}"
5

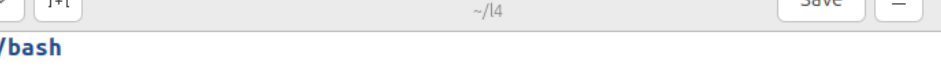
```

LAB 04:

Lab 4

- **String Checking:** Take user input for a word and check if it contains "DIV".
- **Precision and Error Handling:** * Perform division between two numbers and show the output to 3 decimal points.
 - Handle error cases (e.g., division by zero).
- **Grade Generator:** Create a system to generate grades based on marks:

```
ubuntu@ubuntu: ~/l4
ubuntu@ubuntu:~/l4$ gedit word.sh
ubuntu@ubuntu:~/l4$ bash word.sh
Enter a word
abc
Not Same
ubuntu@ubuntu:~/l4$ bash word.sh
Enter a word
awDIU
Not Same
ubuntu@ubuntu:~/l4$ bash word.sh
Enter a word
DIU
Same
ubuntu@ubuntu:~/l4$
```



The screenshot shows a terminal window with a light gray title bar. The title bar contains the text "word.sh" and "~ / l4". On the left side of the title bar are three buttons: "Open", a dropdown arrow, and a file icon. On the right side are three buttons: "Save", a hamburger menu icon, and a close button (X). The terminal content shows a shell script with line numbers 1 through 10. The script uses `echo` to prompt for a word, `read` to store it in variable `w`, and an `if` statement to compare `$w` with `DIU`. Depending on the result, it prints "Same" or "Not Same".

```
1 #!/bin/bash
2
3 echo "Enter a word"
4 read w
5 if [ "$w" == "DIU" ]
6 then
7 echo "Same"
8 else
9 echo "Not Same"
10 fi
```

[illegible]

```
1#!/bin/bash
2
3echo "Enter a word"
4read w
5if [ "$w" == "DIU" ]
6then
7echo "Same"
8elif [ "$w" == "diu" ]
9then
10echo "Same"
11else
12echo "Not Same"
13fi
14
```

The image shows a terminal window and a code editor. The terminal window, titled 'ubuntu@ubuntu: ~/l4', shows the execution of a script named 'division.sh'. The user enters two numbers, and the script outputs the result of the division. The code editor, titled 'division.sh', shows the source code of the script.

```

ubuntu@ubuntu: ~/l4
ubuntu@ubuntu:~/l4$ gedit division.sh
ubuntu@ubuntu:~/l4$ bash division.sh
Enter Two Numbers:
12 3
4.000
ubuntu@ubuntu:~/l4$ bash division.sh
Enter Two Numbers:
12 25
.480
ubuntu@ubuntu:~/l4$ bash division.sh
Enter Two Numbers:
23 -5
DIV not possible
ubuntu@ubuntu:~/l4$

division.sh
~/l4
1 #!/bin/bash
2
3 echo "Enter Two Numbers:"
4 read a b
5
6 if (( $b <= 0))
7 then
8 echo "DIV not possible"
9 else
10 r=$(echo "scale=3; $a/$b" | bc)
11 echo "$r"
12 fi
13

```

The terminal window also shows the execution of a script named 'grade.sh'. The user enters an exam number, and the script outputs the corresponding grade.

```

ubuntu@ubuntu:~/l4$ gedit grade.sh
ubuntu@ubuntu:~/l4$ bash grade.sh
ENTER EXAM NUMBER
100
A+
ubuntu@ubuntu:~/l4$ bash grade.sh
ENTER EXAM NUMBER
60
F
ubuntu@ubuntu:~/l4$ bash grade.sh
ENTER EXAM NUMBER
75
A
ubuntu@ubuntu:~/l4$

```

```

1 #!/bin/bash
2
3 echo "ENTER EXAM NUMBER"
4 read n
5
6 if (( n>=80))
7 then
8 echo "A+"
9 elif (( n>=70))
10 then
11 echo "A"
12 else
13 echo "F"
14 fi

```

LAB 05:

Lab 5

- **Summation and Parity:** Take user input, then calculate the cumulative sum for each number up to n and check if that sum is even or odd.
 - *Example:*
 - n=5
 - 1=1,odd
 - 2=3,odd
 - 3=6,even
 - 4=10,even
 - 5=15,odd

```

ubuntu@ubuntu: ~/l5
ubuntu@ubuntu:~/l5$ gedit loop.sh
ubuntu@ubuntu:~/l5$ bash loop.sh
Enter a number
Thunderbird Mail
1=1, Odd
2=3, Odd
3=6, Even
4=10, Even
5=15, Odd
6=21, Odd
7=28, Even
ubuntu@ubuntu:~/l5$

```

```

Open  loop.sh  Save
~ /l5

1 #!/bin/bash
2
3 echo "Enter a number"
4 read n
5 sum=0
6
7 for (( i=1; i<=n; i++))
8 do
9 sum=$((sum+i))
10 if (( sum%2 == 0 ))
11 then
12 echo "$i=$sum, Even"
13 else
14 echo "$i=$sum, Odd"
15 fi
16 done

```

LAB06:

Lab 6

- **Security:** Password generator using OpenSSL.
- **Logic Tasks:**
 - Reverse a number and check if the reversed number is the same as the original (Palindrome check).
 - Input a 3-digit number and find the sum of its digits.
- **Functions:**
 - Find the factorial of a number using a function call.
 - Add two numbers using a function call.

```

Apr 3 14:24
ubuntu@ubuntu: ~/l6

ubuntu@ubuntu:~$ mkdir l6
ubuntu@ubuntu:~$ ls
Desktop  Documents  Downloads  Music  Pictures  Public  Templates  Videos  a  l2  l3  l4
ubuntu@ubuntu:~$ cd l6
ubuntu@ubuntu:~/l6$ gedit pass.sh
ubuntu@ubuntu:~/l6$ bash pass.sh
Enter the password length:
8
ztdmwV4K
ubuntu@ubuntu:~/l6$ bash pass.sh
Enter the password length:
5
4b4hx
ubuntu@ubuntu:~/l6$ bash pass.sh
Enter the password length:
15
GNAPiEg8iiIAhAx
ubuntu@ubuntu:~/l6$

```



```

pass.sh
~/l6
Open  Save  [Menu]  [Close]

1 #!/bin/bash
2
3 echo "Enter the password length:"
4 read n
5
6 if [[ "$n" =~ ^[0-9]+$ ]]
7 then
8 openssl rand -base64 48 | head -c "$n"
9 echo
10 else
11 echo "enter valid length"
12 fi

```

```

Apr 3 14:29
ubuntu@ubuntu: ~/l6
Firefox
ubuntu:~/l6$ gedit pass.sh
ubuntu@ubuntu:~/l6$ bash pass.sh
Enter the password length:
9
Enter how many pass you want to generate:
6
aNQzdXw4I
e9xClCozB
yiX8MEj6Z
RdDW7d0iG
iKRzV6Hjq
LJS5F5mVt
ubuntu@ubuntu:~/l6$

```

```

pass.sh
~/l6
Open  Save  [Menu]  [Close]

1 #!/bin/bash
2
3 echo "Enter the password length:"
4 read n
5 echo "Enter how many pass you want to generate:"
6 read a
7 if [[ "$n" =~ ^[0-9]+$ ]]
8 then
9 for (( i=1; i<=a; i++))
10 do
11 openssl rand -base64 48 | head -c "$n"
12 echo
13 done
14 else
15 echo "enter valid length"
16 fi

```

```

Apr 3 14:33
ubuntu@ubuntu: ~/l6

ubuntu@ubuntu:~/l6$ gedit pal.sh
ubuntu@ubuntu:~/l6$ bash pal.sh
enter a number:232323
323232
not same
ubuntu@ubuntu:~/l6$ bash pal.sh
enter a number:12221
12221
same
ubuntu@ubuntu:~/l6$

```

```

Open  [v]  [f]  *pal.sh ~/l6  Save  [≡]  [–]  [□]

1 #!/bin/bash
2 read -p "enter a number:" n
3 r=$(echo $n | rev)
4 echo "$r"
5 if ((n == r ))
6 then
7 echo "same"
8 else
9 echo "not same"
10 fi

```

```

Apr 3 14:41
ubuntu@ubuntu: ~/l6

ubuntu@ubuntu:~/l6$ gedit sum.sh
ubuntu@ubuntu:~/l6$ bash sum.sh
enter 3 numbers:
3 4 5
12
enter 3 numbers:
9 7 6
22
ubuntu@ubuntu:~/l6$

```

```

Open  [v]  [f]  sum.sh ~/l6  Save  [≡]  [–]  [□]

1 #!/bin/bash
2 sum(){
3 echo "enter 3 numbers: "
4 read a b c
5 add=$(( a+b+c ))
6 echo "$add"
7 }
8
9 sum
10 sum

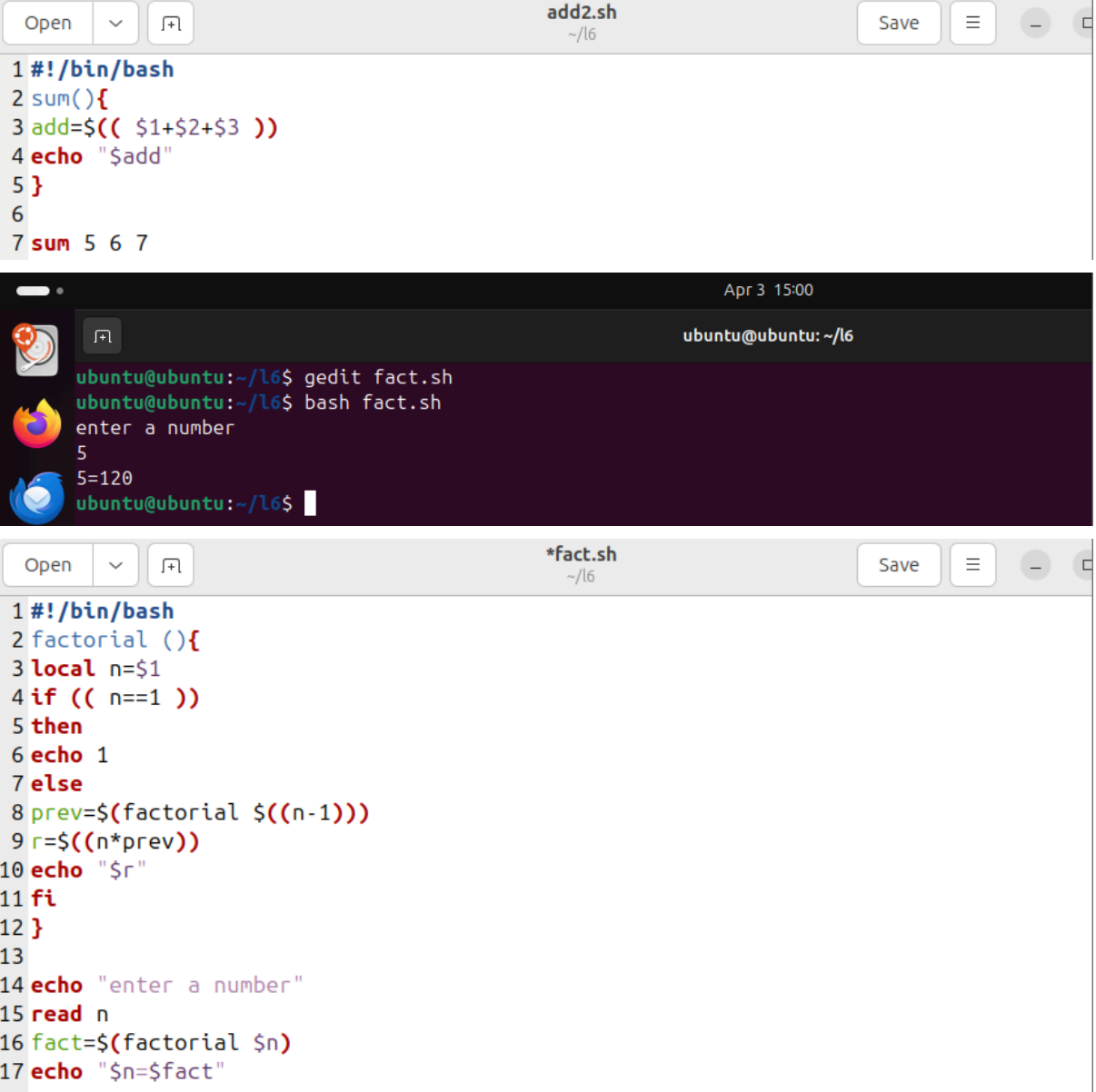
```

```

Apr 3 14:44
ubuntu@ubuntu: ~/l6

ubuntu@ubuntu:~/l6$ gedit add2.sh
ubuntu@ubuntu:~/l6$ bash add2.sh
18
ubuntu@ubuntu:~/l6$

```



The image shows a code editor with three open files. The top file, `add2.sh`, contains a shell script for calculating the sum of three numbers. The middle file is a terminal window showing the execution of `fact.sh` with input 5, resulting in the output 120. The bottom file, `*fact.sh`, contains a shell script for calculating the factorial of a number.

```

1 #!/bin/bash
2 sum(){
3 add=$(( $1+$2+$3 ))
4 echo "$add"
5 }
6
7 sum 5 6 7

```

```

ubuntu@ubuntu: ~/l6
ubuntu@ubuntu:~/l6$ gedit fact.sh
ubuntu@ubuntu:~/l6$ bash fact.sh
enter a number
5
5=120
ubuntu@ubuntu:~/l6$

```

```

1 #!/bin/bash
2 factorial (){
3 local n=$1
4 if (( n==1 ))
5 then
6 echo 1
7 else
8 prev=$(factorial $((n-1)))
9 r=$((n*prev))
10 echo "$r"
11 fi
12 }
13
14 echo "enter a number"
15 read n
16 fact=$(factorial $n)
17 echo "$n=$fact"

```